

COMMAND LABS

CONTENT IDEATION · THE EXPLAINER

Repeatable on YouTube. Rising in search.

How the content ideation system picks video topics, step by step,
and the first principles that explain why every step is there.

A video is a bet. Check the odds twice.

Every video costs you days. Before you spend them, two questions decide whether the topic is worth it, and each one can be answered with public data instead of a feeling.

YouTube Data API

1. Is it repeatable?

Has this topic produced a hit for **many different channels**, recently, including channels about your size? Or did it work once, for one creator, for reasons you can't copy?

Google Ads Keyword Planner

2. Is it rising?

Are **more people looking for it** than a year ago? Or are you arriving at a topic whose audience has already peaked and is walking away?

The system answers both, for every topic, and gives the verdict **make it** only to topics that pass both. This is the scoreboard of a real run, started from the seed keywords "ai agents" and "claude code":

TOPIC	REPEATABILITY	CHANNELS	SEARCHES / MO	YOY	VERDICT
Claude Code	3.45	13	550,000	+406%	make it
AI Agents	2.80	13	49,500	-3%	make it
Code Geass	-	-	135,000	+16%	skipped: unrelated

"Code Geass" is an anime. Google suggested it because it contains the word "code". The system dropped it before spending a YouTube search on it. Page 11 explains how. "AI Agents" passes as rising although its year is down 3%. Page 9 explains why that is the right call.

THE PRINCIPLE BEHIND THE WHOLE SYSTEM

Two independent kinds of evidence that agree are worth more than any amount of one kind.

YouTube performance and Google search demand are measured by different companies, from different behaviour, with different errors. A topic can fool one of them. It rarely fools both.

What actually makes a video get views?

Strip it down and a video's views are the product of four things. Only one of them is the thing you are trying to choose.

$$\text{views} \approx \text{channel} \times \text{topic} \times \text{packaging} \times \text{luck}$$

FACTOR	WHAT IT IS	CAN YOU COPY IT?
Channel	Subscribers, watch history, how hard the algorithm already pushes this creator. It explains most of the raw number.	No. A channel with 2M subscribers can get 200,000 views on an ordinary video.
Topic	How many people want a video about this, right now.	Yes. This is the factor you are shopping for.
Packaging	Title, thumbnail, hook, quality of the video itself.	Partly. It is your craft, not the topic's merit.
Luck	Timing, a news event, one big share.	No.

Seen this way, the usual ways of finding ideas each confuse the topic with another factor:

METHOD	WHAT GOES WRONG
Copy the videos with the most views	Raw views mostly measure the channel . You end up copying big creators' ordinary videos.
Copy one viral video	One data point can't separate topic from packaging and luck . And you only see the winners: the hundred videos on the same topic that flopped are invisible.
Use a keyword tool alone	Search volume proves people want an answer, not that they want a video , and not that videos on it outperform.
Trust your gut	No measurement at all, so no way to learn when it was wrong.

SO THE JOB IS

Isolate the topic factor. Two moves do it.

Normalize: divide every video's views by what its own channel normally gets. That cancels the channel.

Replicate: demand the same result from many unrelated channels. Packaging and luck differ from channel to channel, so they average out. What is left is the topic.

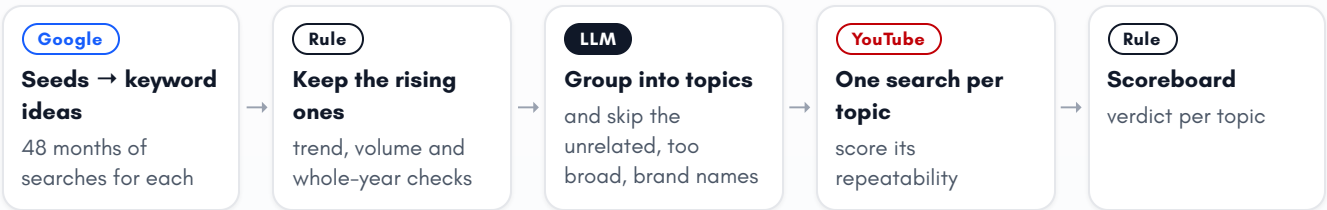
Two pipelines, one scoreboard.

A pipeline is a list of small steps. Each step takes something in, does one job, and hands something on. There are only four kinds of step, and two of them never touch each other.

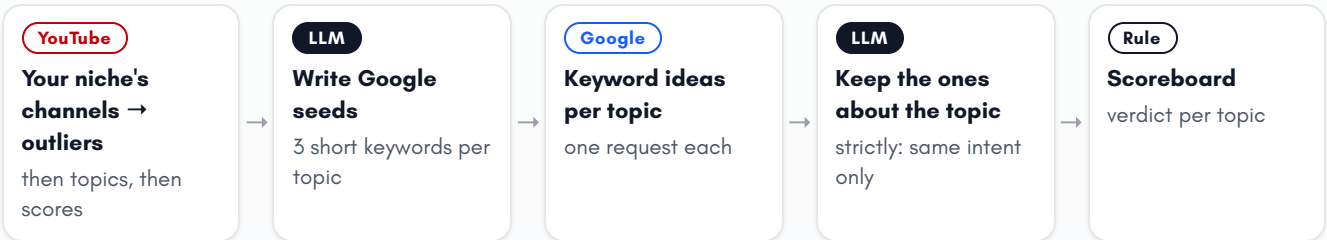
KIND OF STEP	THE QUESTION IT ANSWERS	WHERE THE ANSWER COMES FROM
YouTube	Has this topic worked for many different channels?	YouTube Data API v3: search , videos , channels , playlistItems , plus YouTube's search autocomplete
Google	Are more people searching for this than a year ago?	Google Ads API: GenerateKeywordIdeas (Keyword Planner)
LLM	Which phrasings are the same topic? What would a viewer type?	OpenAI or Anthropic, with answers forced into a fixed structure
Rule	Is this keyword rising? Does this topic pass both tests?	Plain arithmetic in the code. No API, no AI

You can start from either side

PIPELINE 1, FROM KEYWORDS · `UV RUN PIPELINE1_FROM_KEYWORDS.PY "AI AGENTS" "CLAUDE CODE"`



PIPELINE 2, FROM YOUTUBE · `UV RUN PIPELINE2_FROM_YOUTUBE.PY`



WHY THE TWO SIDES ARE KEPT APART

Independence is the point, so it is built into the structure.

The YouTube steps and the Google steps share no code and no data. They only meet in the last step, the scoreboard. So the two kinds of evidence can't contaminate each other, and every step can be run and checked alone. Part 2 shows how.

One number drives everything: the median.

Before any step runs, every channel the pipeline touches gets a baseline: the **median views of its last 50 uploads**, in one format. Every video is then judged against its own channel's baseline, never against a raw view count.

channel median = median of the views of the last UPLOADS_FOR_MEDIAN uploads, in FORMAT
multiple = video views ÷ channel median

FIRST PRINCIPLE: NORMALIZE

Dividing by the channel's median cancels the channel.

In $\text{views} \approx \text{channel} \times \text{topic} \times \text{packaging} \times \text{luck}$, the median is a direct estimate of the channel term: what this creator gets on an ordinary day. Divide by it and what remains is how much *this particular video* beat the ordinary. That is why **20,000** views on a channel that normally gets **2,000** (a 10x) is a stronger signal than **200,000** on a channel that normally gets **150,000** (a 1.3x).

Why the median and not the average

A channel's views are lopsided: many ordinary videos and an occasional monster. The average gets dragged toward the monster. The median doesn't move.

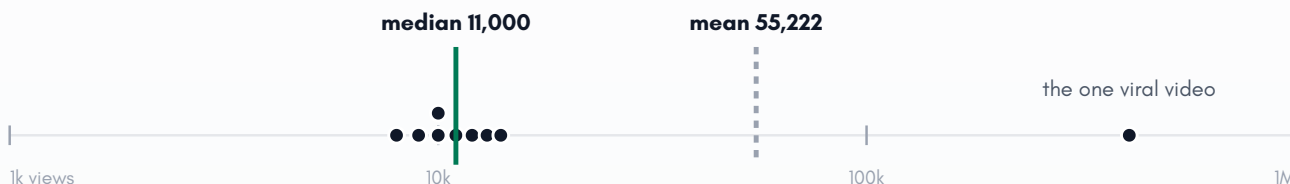


Illustration: nine uploads of one channel, on a log scale. Eight sit between 8,000 and 14,000 views. One got 410,000.

The average says a normal video here gets **55,222** views. Not one of the eight normal videos comes near that. The median says **11,000**, which is exactly what a normal video gets. Measured against the median the viral video is a **37x**. Measured against the average it would be a **7x**, and worse: the viral video would be hiding inside its own baseline, making itself look less exceptional.

Two details that keep the baseline honest

- **Formats are never mixed.** Shorts and long-form videos live on different view scales. A baseline that blends them describes neither, so **FORMAT** picks one and the other is ignored everywhere.
- **Only recent uploads count.** The last 50 uploads describe the channel as it is now. A channel that grew 10x last year should not be judged against what it got before.

Find the outliers. Then find the topics.

Outliers are hypotheses, not answers

The pipeline scans every channel in `CHANNELS_TO_SCAN` and keeps the uploads with at least **5x** their channel's median (`OUTLIER_MULTIPLE`). The **10** biggest multiples go on (`TOP_CANDIDATES`).

FIRST PRINCIPLE: AN ANOMALY NEEDS A CAUSE

A 5x video means something other than the channel did the work.

The channel is already cancelled, so the cause is the topic, the packaging or luck. One video can't tell you which. So finding outliers proves nothing. It produces a short list of **hypotheses**: "maybe this topic pulls views by itself". The repeatability step is the experiment that tests them.

This is also why the list of channels matters so much. It is the only place hypotheses come from, so the results can never be more relevant to your niche than these channels are.

The unit that repeats is the topic, not the title

Several outliers are often about the same thing, and a title is packaging. "I Built a \$10k AI Agent in 6 Hours" is one creator's wrapping. The thing another creator can repeat is "building and selling AI agents". So an LLM reads the outlier titles, groups the ones about the same underlying topic, and writes for each topic **the search query a viewer would type into YouTube**: short, generic, no channel names, no clickbait.

Then each query goes to **YouTube's own search autocomplete**, which returns the phrasings viewers really type, most popular first. It costs no API quota. The LLM picks up to **3** of them (`QUERY_VARIANTS`): same language, still on topic, a distinct angle such as "...for beginners". Every pick is scored as an idea of its own.

FIRST PRINCIPLE: USE EACH TOOL FOR WHAT IT IS RELIABLE AT

The LLM judges language. It never produces a number.

"Are these two titles about the same thing?" is a language question, and LLMs are good at it. "How well does this topic perform?" is a measurement, and LLMs invent measurements. So every score in this system is plain arithmetic on API data, and the LLM is fenced in three ways: its answers must fit a **fixed structure**, its autocomplete picks are **checked against the real suggestion list** (an invented query is thrown away), and its answers are **cached** for a day, so re-running gives the same topics.

Repeatability: run the experiment.

For each query, the pipeline searches YouTube and asks one thing of the results: how many **other** channels also beat their own median with this topic?

FIRST PRINCIPLE: REPLICATION

If the topic is the cause, the effect shows up wherever the topic does.

If the outlier happened because of that creator's thumbnail, or luck, other channels covering the topic will have performed normally, and the search finds no hits. If the topic itself pulls views, then channel after channel, with different thumbnails, skill and timing, will show a video that beat its baseline. Packaging and luck differ for every channel, so across many channels they cancel. The topic is the only thing they share.

What counts as a hit

hit = video views $\geq$ HIT_MULTIPLE $\times$ its own channel's median 4x
and video views $\geq$ MIN_HIT_VIEWS 1,000 views
and age $\leq$ MAX_HIT_AGE_DAYS 730 days

RULE	THE REASONING
4x its own median	The same normalization as for outliers: a hit is measured against the channel that made it.
At least 1,000 views	On a dead channel with a median of 4 views, a video with 17 is a "4x". A ratio means nothing when the numbers under it are noise.
At most 2 years old	Views pile up for years, but the median describes the channel today. An old video's lifetime views can't be compared with today's baseline.
Your own channel is left out	Your own past hit is not independent evidence about whether it works for others.
The first 20 different channels	Every channel examined costs quota for its baseline. CHANNELS_PER_QUERY caps the bill at the most relevant results.

FIRST PRINCIPLE: COUNT INDEPENDENT WITNESSES

Five hits from one channel are one piece of evidence. Five channels with one hit each are five.

A creator who is excellent at a topic will hit with it again and again. That tells you about the creator. So a topic only qualifies with hits from **6+ different channels** (MIN_HITS). In the run on page 2, "claude code" had **22** hits across **13** channels.

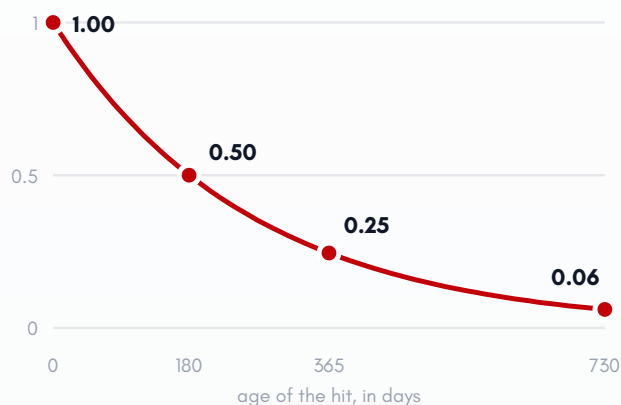
Not every hit is equally good evidence.

Counting hits would treat a three-week-old hit from a channel like yours the same as a two-year-old hit from a channel a thousand times your size. So each hit is weighed by how much it says about **you, now**.

recency	= $0.5 ^ { \text{age in days} \div \text{AGE_HALF_LIFE_DAYS} }$	half-life: 180 days
size gap	= $ \log_{10}(\text{their median}) - \log_{10}(\text{your median}) $	
weight	= $\text{recency} \div (1 + \text{size gap})$	
repeatability score	= the sum of the weights of all the topic's hits	

Recency: evidence goes stale

A hit from last month says the audience is there today. A hit from 18 months ago says it was there once. A half-life is the simplest honest model of that: every **180** days, a hit counts half as much.



Size: evidence transfers between similar channels

What works for a channel 1,000x your size may depend on an audience you don't have. The gap is measured in **powers of ten**, because channels differ multiplicatively: 1k to 10k views is the same kind of jump as 100k to 1M.

THEIR MEDIAN VS YOURS	SIZE FACTOR
The same	1.00
10x bigger or smaller	0.50
100x	0.33
1,000x	0.25

It falls slowly on purpose: a big channel's hit is weaker evidence, not worthless evidence.

Reading the score. A brand-new hit from a channel exactly your size weighs **1.0**, so a score of **3.45** is "evidence worth about three and a half perfect hits". A topic qualifies with a score of **1.0+** (**MIN_SCORE**) and hits on **6+** channels: the channel count blocks one prolific creator, the score blocks a pile of old hits from giant channels.

Rising means the floor is rising.

Keyword Planner returns how often a keyword was searched in each of the last **48 months**. A rule turns that history into one word: rising, flat, shrinking or new. Two problems have to be solved first.

PROBLEM 1: SEASONS

Compare a year with a year.

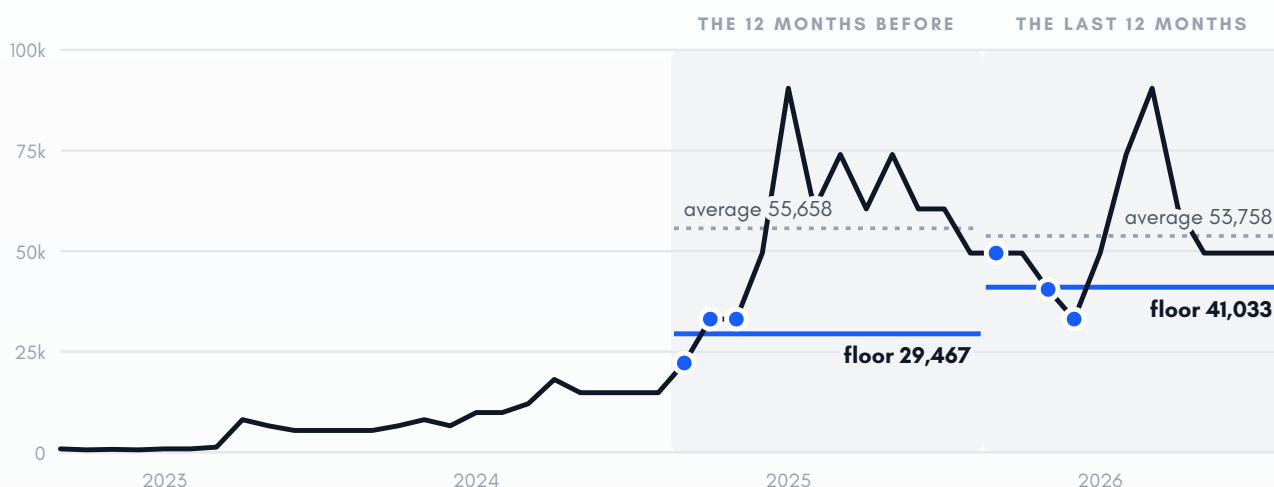
"Tax software" rises every spring. Comparing this month with last month measures the calendar. So every comparison is the **last 12 months against the 12 before**: both windows contain every calendar month exactly once, and the seasons cancel.

PROBLEM 2: SPIKES

Measure the quiet months.

One news event can double a year's average without one extra person caring a month later. So the trend comes from the **floor**: the mean of the 3 quietest months of a window. It is the demand that is there even when nothing is happening.

A real example: "ai agents"



Monthly Google searches for "ai agents" in the US. Blue dots are the 3 quietest months of each window, the blue line is their mean (the floor), the dotted line is the window's average.

By the average, the two years look the same: **down 3%**. Each contained a spike to about 90,000 searches, and spikes dominate an average. Now look at the quiet months. A year ago the slowest months had **29,467** searches. Now the slowest months have **41,033: +39%**. The demand that is there when nothing special is happening has grown by more than a third. A video serves that audience, so it is called **rising**.

The labels. **rising**: the floor is up 20% or more. **shrinking**: down 20% or more. **flat**: in between. **new**: there was no floor a year ago, and there are searches now.

Know how blunt your instrument is.

Keyword Planner does not report exact counts. It rounds every volume into buckets roughly 20 to 25% apart: 40,500, then 49,500, then 60,500. That single fact sets two thresholds.

FIRST PRINCIPLE: A CHANGE SMALLER THAN YOUR MEASUREMENT ERROR IS NO CHANGE

Why "rising" starts at +20%, and why tiny keywords are ignored.

When a keyword moves from one bucket to the next, the reported change is about 20% even if the true change was 2%. So anything under **20%** is called flat. For the same reason, keywords under **200** searches a month (`MIN_MONTHLY_SEARCHES`) are dropped: down there the buckets are 10, 20, 30, and one step is a "50% rise".

The full "rising" rule

A keyword goes on only when all three hold:

- its trend is **rising** or **new** (`KEEP_TRENDS`),
- it has at least **200** searches a month,
- and its searches over the **whole year** are not down 20% or more (`YOY_FALLING_PCT`).

The third check is the floor's safety net. A keyword can have a rising floor inside a collapsing year: the quiet months crept up while the total fell off a cliff. That is a topic past its peak. "ai agents" at -3% is nowhere near that line.

What the filter does to a real run

139

keyword ideas returned by Google for 2 seeds

11

are rising, with enough volume to trust

3

topics, after the LLM groups the phrasings

2

YouTube searches spent: 200 of 10,000 daily quota units

Of the 128 dropped keywords, 126 had under 200 searches a month, 1 was flat and 1 was shrinking. Every reason is saved to a file.

FIRST PRINCIPLE: SPEND THE SCARCE RESOURCE LAST

Keyword lookups are free. YouTube searches are not.

A search costs **100** of your **10,000** daily quota units. So the cheap filters run first. "AI Agents" alone held **9** phrasings: searched one by one, 900 units for a single idea. Grouped into a topic, 100.

YouTube and Google don't speak the same language.

The first version of this system matched the two tools on the exact phrase. It rarely worked, and the reason is worth understanding.

On YouTube

People browse. They type long, specific, format-flavoured phrases:
"build and sell ai agent 6 hours course"

On Google

People ask. They type short noun phrases, and Keyword Planner only knows those:
"ai agent course"

Same topic, same audience, zero overlap in wording. Looked up literally, the YouTube phrase has no Google data at all, and a great topic is scored "no search volume".

FIRST PRINCIPLE: JUDGE THE THING, NOT ITS LABEL

A topic is judged as a whole, whatever phrasing each side knows it under.

On YouTube a topic is represented by its **best-scoring phrasing**. On Google it is represented by its **most searched keyword**. Deciding which phrasings belong to the same topic is a language judgement, so it is the LLM's job.

What the LLM is asked, in each direction

DIRECTION	THE LLM'S TASK	THE GUARD RAIL
From keywords	Group the rising keywords into video topics. Name each topic and write its YouTube search query. Mark a topic to skip when a search would teach nothing: <i>unrelated</i> ("code geass"), <i>too broad</i> ("ai"), or a bare <i>brand name</i> ("chatgpt").	It must copy keywords exactly. A keyword it invents is discarded, and no keyword can land in two topics.
From YouTube	First: write 3 short Google seed keywords for each repeatable topic. Then, from what Google suggests back: keep only the keywords of people who want the same thing as the YouTube viewers.	It is told to be strict. One big, loosely related keyword would otherwise decide the whole topic's trend.

Two smaller details. Keyword Planner counts close variants ("ai agent", "ai agents") together and reports the identical 48-month history for each. The pipeline spots identical histories and treats them as one keyword, so searches are never counted twice. And starting from YouTube, Keyword Planner is asked **once per topic**: in a shared request, one big topic's suggestions crowd out the rest.

Your cue to double-check. The scoreboard reports **rising_keywords**, such as "3 of 8". When only a few are rising, the verdict rests on the biggest keyword alone, so check that it really is the topic.

Both tests, or no.

Every topic lands in one of four boxes. Only one of them says make it.

	RISING ON GOOGLE	NOT RISING
REPEATABLE ON YOUTUBE	make it Proven format, growing audience. It works for channels like yours, and more people want it every month.	"repeatable, but search is shrinking" It works today, for an audience that is leaving. You would be late. The verdict names the exact reason.
NOT REPEATABLE	"rising, but not repeatable" People want it, but videos on it don't outperform. Often the demand is for an answer, a tool or a website, not for a video.	"not repeatable" Nothing here.

FIRST PRINCIPLE: DIFFERENT FAILURE MODES NEED SEPARATE GATES

Why it is a strict AND, not a blended score.

It is tempting to average the two numbers into one. But they guard against different mistakes, and a blend lets a huge value on one side buy forgiveness on the other. A topic with 2,000,000 searches and no repeatability is not "pretty good". It is a topic where videos don't win. Two gates can't be traded against each other.

How a topic is judged on each side

TEST	THE TOPIC IS JUDGED BY	IT PASSES WHEN
Repeatable	its best-scoring phrasing on YouTube	hits on MIN_HITS + different channels and a score of MIN_SCORE +
Rising	its most searched Google keyword	trend rising or new, MIN_MONTHLY_SEARCHES + a month, whole year not down YOY_FALLING_PCT or more

Two more verdicts exist for topics that never reached YouTube: "skipped: ..." with the LLM's reason, and "not searched on YouTube: outside the 10 biggest", when more topics qualified than MAX_TOPICS allows. The ones that do go are those that gained the most searches on the year before, so the quota is spent on topics that are both big and growing.

Within the scoreboard, "make it" topics come first, highest repeatability on top. Repeatability breaks the tie because it is the evidence closest to what you actually do: publish on YouTube.

Every step: what goes in, what comes out, how to try it alone.

You don't need to read code to follow a run. Each step is one small file in the **steps** folder, and a pipeline file is just the steps in order, with the input and output written above every line.

```
# pipeline1_from_keywords.py, steps 2 and 3, exactly as they appear in the file

run.step("Is each keyword rising, flat or shrinking?")
# in: {"keyword": "ai agents", "monthly_searches": {"2022-09": 880, ...}}
# out: {"keyword": "ai agents", "yoy_change_pct": -3.4, "floor_change_pct": 39.3, "trend": "rising", ...}
keywords = search_trends(ideas)
run.save("search_trends", input=ideas, output=keywords)

run.step("Which keywords are rising, with enough searches to trust?")
# in: 139 keywords with a trend
# out: the same keywords, each with a verdict: "goes on", or the reason it doesn't
judged = rising_keywords(keywords)
run.save("rising_keywords", input=keywords, output=judged)
```

Three ways to look at any step

WAY

HOW

Run it on its own

Every step has a one-line command, shown in the dark bar of its card on the next pages. It runs that step alone on a small example and prints the result. Steps marked **Rule** need no API key at all, so you can try them the minute you have installed **uv**.

Read its file from a run

After a run, open **runs/<run id>/**. There is one file per step, numbered in order: **02_search_trends.json** holds what went **in** to step 2 and what came **out**.

Click through the viewer

Open **viewer/index.html**. Same steps, same order, with tables, charts, thumbnails and a plain-language note on what each step found.

FIRST PRINCIPLE: SMALL PARTS YOU CAN CHECK BEAT ONE BIG PART YOU MUST TRUST

If a verdict looks wrong, exactly one step is wrong, and you can find it.

Each step does one job and hands on plain data. So a surprising result is never a mystery. You walk back through the files until a step's output stops making sense, run that one step alone, and look.

The commands assume you are in the project folder with **.env** and **config.py** set up (last page). The **curl** lines call the same web address the step calls, so you can see the raw answer. They expect your key in the terminal: **export YOUTUBE_API_KEY=your-key**

Start from what people search.

A real run: `uv run pipeline1_from_keywords.py "ai agents" "claude code"`

1 Keyword ideas Google

steps/keyword_ideas.py

Asks Google Ads Keyword Planner which keywords people search around your seeds. Every keyword comes back with 48 months of search history.

IN

```
["ai agents", "claude code"]
```

OUT

```
139 keywords, such as { "keyword": "claude code",
"avg_monthly_searches": 550000, "monthly_searches": { "2022-09": 320,
... "2026-08": 301000 } }
```

TRY IT `uv run python -m steps.keyword_ideas "ai agents"` prints the 10 most searched. Needs Google Ads set up

2 Search trends Rule

steps/search_trends.py

Turns each history into a trend: the last 12 months against the 12 before, measured on the **floor**, the 3 quietest months (page 9).

IN

```
{ "keyword": "ai agents", "monthly_searches": {
"2022-09": 880, ... } }
```

OUT

```
{ "keyword": "ai agents", "yoy_change_pct": -3.4, "floor_prior_12m":
29467, "floor_last_12m": 41033, "floor_change_pct": 39.3, "trend": "rising"
}
```

TRY IT `uv run python -m steps.search_trends` no API key needed: a made-up keyword whose spike hides its growth

3 Rising keywords Rule

steps/rising_keywords.py

Gives every keyword a verdict: it goes on, or the reason it doesn't. Rising means: trend rising or new, 200+ searches a month, whole year not down 20%.

IN

139 keywords, each with its trend

OUT

```
"claude code" → goes on · "ai agents" → goes on · ...
11 go on. 126 × under 200 searches a month, 1 × search is flat, 1 × search is
shrinking
```

TRY IT `uv run python -m steps.rising_keywords` no API key needed: three keywords, three different verdicts

4 Group into topics LLM

steps/group_into_topics.py

An LLM groups the keywords that are the same video topic and writes the YouTube search for each. It marks topics not worth a search.

IN

the 11 rising keywords, with their searches a month

OUT

```
{ "name": "AI Agents", "search_query": "ai agents", "keywords": [9
phrasings], "skip": "" }
{ "name": "Claude Code", "search_query": "claude code", "keywords":
["claude code"], "skip": "" }
{ "name": "Code Geass", "search_query": "code geass", "keywords":
["codegeass"], "skip": "unrelated" }
```

TRY IT `uv run python -m steps.group_into_topics` one small LLM call on four example keywords

Then ask YouTube, once per topic.

5 Choose topics Rule

steps/choose_topics.py

A YouTube search costs 100 of your 10,000 daily quota units. So at most **MAX_TOPICS** topics go on: the ones whose biggest keyword gained the most searches on the year before. Skipped topics never go on.

IN

the 3 topics, and the keywords with their trends

OUT

Claude Code: gained +405,114 searches a month → goes on
 AI Agents: gained -1,900 → goes on (its year is flat, its floor is rising)
 Code Geass: skip "unrelated" → does not go on

TRY IT `uv run python -m steps.choose_topics` no API key needed

6 Score repeatability YouTube

steps/score_repeatability.py

Searches YouTube for each query. A result is a **hit** when it has 4x its own channel's median views. Hits weigh more when recent and from channels near your size. The score is the sum of the weights (pages 7 and 8).

IN

```
[ { "search_query": "claude code", "topic":
  "Claude Code" },
  { "search_query": "ai agents", "topic": "AI
  Agents" } ]
```

OUT

```
{ "search_query": "claude code", "score": 3.44, "hit_channels": 13, "hits": [
  22 videos, the heaviest first:
  { "title": "How to Build an AI Agent with Claude Code...", "views": 449513,
    "channel_median": 9909, "multiple": 45.4, "age_days": 83, "weight":
    0.256 }, ... ] }
```

TRY IT `uv run python -m steps.score_repeatability "claude code"` 100 quota units, free when cached

RAW CALL `curl "https://www.googleapis.com/youtube/v3/search?
 part=id&type=video&maxResults=5&q=claude+code&key=$YOUTUBE_API_KEY"`

7 Scoreboard Rule

steps/scoreboard.py

Joins the two sides, topic by topic. On YouTube the topic's best-scoring phrasing counts, on Google its most searched keyword. A topic that is repeatable **and** rising gets "make it" (page 12).

IN

the topics, the keywords with their trends, the YouTube scores

OUT

Claude Code: **make it** (3.44 on 13 channels · 550,000 searches a month · rising)
 AI Agents: **make it** (2.80 on 13 channels · 49,500 · rising)
 Code Geass: skipped: unrelated

TRY IT `uv run python -m steps.scoreboard` no API key needed

WHAT THIS RUN COST

139 keywords in, 2 YouTube searches out.

Only step 6 spends YouTube quota, and steps 3 to 5 exist so it is asked as few questions as possible: **200** of your **10,000** daily units here.

Start from what already worked.

A real run: `uv run pipeline2_from_youtube.py` with 24 AI and tech channels in `CHANNELS_TO_SCAN`

1 Find outliers

YouTube
steps/find_outliers.py

Gives every channel a baseline, the median views of its last 50 uploads (page 5), and keeps the uploads with 5x that or more. The 10 biggest go on.

IN

["@Fireship", "@t3dotgg", ...] the channels of your niche

OUT

10 outliers, such as
 { "title": "Convert Image PDF to Editable Text in 5 Steps...", "views": 6091050,
 "channel_median": 28012, "multiple": 217.4 }

TRY IT `uv run python -m steps.find_outliers @Fireship @TechWithTim` about 3 quota units a channel

RAW CALL `curl "https://www.googleapis.com/youtube/v3/channels?part=id&forHandle=@Fireship&key=$YOUTUBE_API_KEY"`

2 Outliers to topics

LLM
steps/outliers_to_topics.py

A title is packaging. An LLM groups the outliers that are about the same topic and writes one short, generic YouTube search per topic.

IN

the 10 outliers: title, channel, multiple

OUT

{ "name": "AI Tools and Applications", "search_query": "AI tools and agents", "video_ids": [8 videos] }
 { "name": "PDF Conversion and Compression", "search_query": "PDF to text conversion", "video_ids": [2 videos] }

TRY IT `uv run python -m steps.outliers_to_topics` one small LLM call on three example titles

3 Query variants

YouTube
LLM
steps/query_variants.py

Asks YouTube's search box what viewers really type (free, no quota). An LLM picks up to 3 per topic: still on topic, a distinct angle. A pick that is not in YouTube's list is thrown away.

IN

the 2 topics with their search query

OUT

6 queries to score, such as
 { "search_query": "pdf to text converter ai", "topic": "PDF Conversion and Compression" }
 { "search_query": "ai tools and agents podcast", "topic": "AI Tools and Applications" }

TRY IT `uv run python -m steps.query_variants "claude code"`

RAW CALL `curl "https://suggestqueries.google.com/complete/search?client=firefox&ds=yt&hl=en&q=claude+code"` no key

Score them, then ask Google in Google's words.

4 Score repeatability

YouTube

steps/score_repeatability.py

The same step as step 6 of pipeline 1, now with six queries. Each topic is later judged by its best-scoring phrasing.

IN

the 6 search queries

OUT

2.95 · 20 hits on 13 channels · "AI tools and agents" → repeatable
0.75 · 5 hits on 5 channels · "pdf to text converter ai" → not repeatable
and four more, all lower

TRY IT

```
uv run python -m steps.score_repeatability "pdf to text converter ai"
```

5 Google seeds

LLM

steps/google_seeds.py

Nobody googles a YouTube phrasing. An LLM writes 3 short seed keywords per topic, the way people would type it into Google (page 11).

IN

```
{ "name": "PDF Conversion and Compression",  
  "phrasings": ["pdf to text converter ai", ... ] }
```

OUT

```
"seeds": [ "pdf to text converter", "pdf to text compression", "convert PDF  
to text" ]
```

TRY IT

```
uv run python -m steps.google_seeds one small LLM call
```

6 Keyword ideas per topic

Google

Rule

steps/keyword_ideas.py + search_trends.py

Steps 1 and 2 of pipeline 1, run once per topic: in one shared request, a big topic's suggestions crowd out the rest. Of what Google suggests, the 40 biggest with 200+ searches a month go on.

IN

each topic's 3 seeds

OUT

PDF topic: 2,392 suggestions → the 40 biggest go on
AI tools topic: 2,131 suggestions → the 40 biggest go on

TRY IT

```
uv run python -m steps.keyword_ideas "pdf to text converter" "convert PDF to text"
```

7 Pick topic keywords

LLM

steps/pick_topic_keywords.py

Google's suggestions are only loosely related. An LLM keeps the keywords of people who want the same thing as the YouTube viewers, and is told to be strict.

IN

each topic with its 40 candidates

OUT

PDF topic: 22 of 40 kept ("ocr a pdf", "pdf into text", "pdf to txt" ...)
AI tools topic: 9 of 40 kept ("ai tools", "free ai tools", "ai app" ...)

TRY IT

```
uv run python -m steps.pick_topic_keywords one small LLM call: it should drop "edit pdf"
```

The verdict, and which pipeline to use.

8

Scoreboard

Rule

steps/scoreboard.py

The same step as step 7 of pipeline 1: repeatable on YouTube **and** rising on Google, or no.

IN

the topics, their keywords with trends, the YouTube scores

OUT

AI Tools and Applications: **repeatable, but search is flat**
PDF Conversion and Compression: **not repeatable**

TRY IT

uv run python -m steps.scoreboard no API key needed

Which pipeline, when?

	PIPELINE 1, FROM KEYWORDS	PIPELINE 2, FROM YOUTUBE
You bring	A few seed keywords about your niche	The channels of your niche, in config.py
Good at	Finding topics nobody in your niche has made yet, because it starts from what people want	Finding formats that are proven right now, because it starts from what already won
Blind to	Topics people watch but never google	Anything the scanned channels haven't tried
YouTube quota	Low: one search per topic, at most 1,000 units	Higher: up to 4 searches per topic, at most 4,000 units
Needs Google Ads	Yes, from the first step	Only from step 6. Without it, the run stops after step 4 with the repeatability scores

READ THIS RUN: NOTHING TO MAKE, AND THAT IS THE SYSTEM WORKING

A 217x viral video was rejected. So was a keyword up 632%.

The biggest outlier, a PDF tutorial with **6,091,050** views, produced a topic with hits on only **5** channels: it worked for one creator, not for everyone. And the AI tools topic is repeatable, but its keyword "free ai tools" is up **632%** on the year with a **flat floor**: one spike, no lasting growth (page 9). Both would have fooled a simpler method.

Built so you can check its work.

A system that hands you a verdict and hides its reasoning deserves no trust. Every decision this one makes is written down where you can read it.

DESIGN CHOICE	WHAT IT GIVES YOU
One folder per run	A run is named by its start time, like <code>20260921-123241</code> , and everything it produced is in <code>runs/20260921-123241/</code> , including every setting it ran with.
One file per step	<code>01_a_seeds_to_keywords.json</code> , <code>02_search_trends.json</code> , and so on. Each holds what went in and what came out . When a verdict surprises you, you can walk back to the exact step that caused it.
The run viewer	Open <code>viewer/index.html</code> . Each step shows what it does, what it found in this run, and its data, down to the thumbnails of the hit videos behind a score. A run that failed shows how far it got.
A 24-hour cache	YouTube and LLM responses are stored on disk. Repeating a run costs no quota, and changing one setting re-uses everything that setting doesn't touch. <code>--refresh</code> pulls fresh data.
One config.py	Every threshold in this document is a named setting with a comment. Nothing is buried in the code.

FIRST PRINCIPLE: A MODEL YOU CAN'T INSPECT, YOU CAN'T IMPROVE

Every threshold here is a judgement call. You are supposed to argue with them.

Is 4x the right bar for a hit? Is 180 days the right half-life for your niche? Nobody knows in advance. Because every step's output is saved, you can change a setting, re-run from the cache for free, and see exactly which topics moved and why.

What a run costs

RESOURCE	COST
YouTube Data API	Free, 10,000 quota units a day. A search is 100 units, everything else is 1. From keywords: at most <code>MAX_TOPICS</code> searches, so 1,000 units. From YouTube: up to 10 topics × (1 query + 3 variants) = 40 searches, 4,000 units at the very most. Each channel baseline adds about 2 units.
Google Ads API	Free. You need a Google Ads account and approved API access, which takes a while, so apply first.
LLM	A handful of short calls per run, to a small model by default (<code>gpt-4o-mini</code>).

What it can't tell you.

First-principles thinking includes knowing where your model stops being true. These are the assumptions to keep in mind when you read a scoreboard.

LIMIT	WHAT IT MEANS FOR YOU
YouTube search shows winners	Search ranks videos that already performed, so hits are over-represented in every result list. A score is therefore not a probability of success. Read scores against each other , across topics measured the same way.
Google search is a proxy	YouTube publishes no search volumes. Google demand and YouTube demand usually move together, but not always: some topics are watched without ever being googled.
Your channel list is your horizon	Starting from YouTube, the pipeline can only find topics that the scanned channels have already made. A topic nobody in your list has tried is invisible. Starting from keywords is the way around it.
The LLM can group wrongly	It may attach a big, loosely related keyword to a topic. That is what rising_keywords and the expandable rows in the viewer are for.
A topic is not a video	The system cancels packaging to find topics that win regardless of it. You still have to package yours well. It tells you where to dig, not how.
Location and language	Search data is for the United States in English by default.

The settings most worth tuning

SETTING	TURN IT UP	TURN IT DOWN
CHANNELS_TO_SCAN	A wider pool of hypotheses	Drop channels outside your niche: one off-niche viral video takes a candidate slot
OUTLIER_MULTIPLE (5)	Only true breakouts, more of them one-offs	More candidates, some of them just a good week
HIT_MULTIPLE (4)	Fewer, stronger hits per topic	More hits, weaker proof
AGE_HALF_LIFE_DAYS (180)	Slow niches, where last year's hits still count	Fast niches such as AI news, where 3 months is old
MIN_HITS (6)	Stricter proof of repeatability	Small niches, where 6 channels on one topic is rare
MIN_MONTHLY_SEARCHES (200)	Only topics with a large audience	Niche topics, at the price of noisier trends

What every YouTube setting does in both directions: docs/repeatability-settings.pdf in the repo.

THE WHOLE ARGUMENT, IN SIX LINES

Normalize. Replicate. Weigh. Cross-check.

Normalize	Divide by the channel's median, so you compare topics and not channel sizes.
Replicate	Demand the same result from many unrelated channels, so packaging and luck cancel.
Weigh	Trust recent hits from channels like yours the most.
Measure the floor	Compare year with year and quiet months with quiet months, so seasons and spikes cancel.
Cross-check	Require YouTube and Google to agree, judged per topic and not per phrase.
Show your work	Save every step, so every verdict can be traced and every threshold argued with.

Run it yourself

```
cp .env.example .env          your YouTube and OpenAI keys
cp config.py.example config.py your channel, and the channels of your niche
uv run pipeline1_from_keywords.py "your seed keyword" start from Google search
uv run pipeline2_from_youtube.py start from YouTube
```

The code is in the repo linked in the video description. The README has the full setup, including Google Ads API access.